

Bayridge Secondary School
Pairs Coding Invitational 2025

Problem 1: Doing Laps

You and your friend are playing a board game that is rectangular in shape. The example shown here is 10 x 6 and there are 28 squares in total. The minimum size for a board is 3 x 3 where there would be 8 squares. The maximum size is 15 x 15 where there would be 56 squares. You can assume that the starting point of the board game is the bottom left square.

You are to create a program that calculates the **total number squares traveled AND the number of laps done** given the sequence of rolls by a player. As soon as the player's board piece arrives back at the bottom left square, that is considered one lap.

For *one* case, the first two numbers of input are the length and width of the game board. The next number is the number of rolls by the player, and the following numbers are the values of the rolls by the player. The output should be the total number of squares along with the number of laps. All of this data constitutes one case.

Your program should be able to take in 10 different cases and output 10 different total number of squares and number of laps.

Sample Input for Two Cases

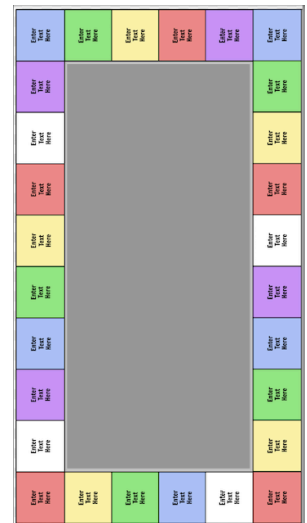
4
5
4
9
11
10
6
14
9
2
13
18

For the first case ... 4 is the length of the board, 5 is the width of the board, 4 is the number of rolls, 9 is the value of roll 1, 11 is the value of roll 2, 10 is the value of roll 3, and 6 is the value of roll 4

Sample Output for Two Cases

36 2
31 0

For the first case ... 36 is number of squares traveled and 2 is number of laps of the board



Problem 2: Secret Messaging

You and your friend decide to create a system where you send "encoded" emails and texts only. To keep things relatively simple, you decide on a system where you will reverse the words and then switch every third character by one (so an 'a' becomes 'b', an 'm' becomes an 'n', and a 'z' becomes an 'a').

Write a program that takes 10 individual words (Strings) as input, reverses them, and "switches" every third character by one (starting from index 2 in the reversed string, which corresponds to every third character from the end of the original word). You can always assume that the input string will only be one word that contains all lowercase letters from a-z, and the words will always have at least 3 characters.

Example Input for Three Strings:

```
hello  
ezme  
programming
```

Expected Output for Three Strings:

```
olmeh  
emae  
gnjmmbrgprp
```

Problem 3: Shuffled Sentences

You won a huge award for computer programming, and you have to go to a banquet to give a speech! You wrote the speech, but a computer virus got into your text file and it shuffled up the words in each sentence. You were able to go through each of the shuffled sentences and tag each word with its original position by putting the position at the end of each word of the sentence. Now you just need to write a program that will **reconstruct the sentence in the correct order**.

The input for this problem will be 10 shuffled sentences, each on their own line. Each word in the sentence has a **number at the end** that represents its original position (1-based index). The sentences will always contain **at least 2 words and at most 10 words** and numbers are **only at the end** of words.

Your job is to print each sentence, rearranged in the correct order, without the numbers. The example below is for only 2 shuffled sentences, but you need to write a program to accept and correct 10 of them.

Example Input for Two Shuffled Sentences:

```
is2 contest5 This1 wicked4 a3  
good3 weaken8 It's1 life4 don't7 you6 a2 if5
```

Expected Output for Two Shuffled Sentences:

```
This is a wicked contest  
It's a good life if you don't weaken
```

Problem 4: Robot Redirection

A robot starts at coordinate (0,0) and follows a sequence of moves given as a string consisting solely of the characters:

- N: move one step North (y increases by 1)
- S: move one step South (y decreases by 1)
- E: move one step East (x increases by 1)
- W: move one step West (x decreases by 1)

After executing the given moves, the robot ends at some final position (x, y).

Your task is to determine the optimal return path—a sequence of moves that brings the robot back to (0,0)—using **the minimum number of moves, which must be fewer moves than the original sequence**.

If such a path exists, output that return path as a String (the optimal valid return path). If no such path exists, output -1.

Input Format:

- A single line containing a string S ($1 \leq \text{length of } S \leq 100$) composed solely of the characters N, S, E, W.

Output Format:

- If it is possible for the robot to return to (0,0) in strictly fewer than |S| moves, output the optimal return path as a string.
- Otherwise, output -1.

Sample Input for Three Different Cases (note that the input strings are not numbered - it's only done here for clarification of the question)

1. NNESSWWSW
2. NESW
3. N

Sample Output for Three Different Cases (with explanation - you don't need to provide explanation)

1. output is **EEN or ENE or NEE** because if I move East twice and then North, the robot is back to the start in, but the order doesn't matter only 3 moves which is less than the original 9
2. output is an **empty string** because the robot is already at the start
3. output is **-1** since it's impossible to find a way back to the start that is fewer moves than 1

Problem 5: I'm Going (Coco)Nuts

Several men (x of them) crash-land their airplane on a deserted island in the South Pacific. On their first day they gather as many coconuts as they can find into one big pile. They decide that, since it is getting dark, they will wait until the next day to divide the coconuts.

By lot they chose 5 among them to watch for rescue searchers while the others slept. The first watcher got bored so he decided to divide the coconuts into x equal piles. When he did this, he found he had one remaining coconut. He gave this coconut to a monkey, took one of the piles, and hid it for himself. Then he jumbled up the $(x-1)$ other piles into one big pile again.

To cut a long story short, each of the five men ended up doing exactly the same thing. They each divided the coconuts into x equal piles and had one extra coconut left over, which they gave to the monkey. They each took one of the x piles and hid those coconuts. They each came back and jumbled up the remaining $(x-1)$ piles into one big pile.

The input for this question contains 10 numbers on 10 different lines representing the number of men that crash land (x). Each number is between 5 and 50.

Write a program that will find the **smallest number of coconuts that could have been in the original pile** for each of the 10 instances.

Sample Input for One Example:

6

Sample Output for One Example:

7771

Problem 6: Rolling Out the Ones

"Nerd Poker" is a podcast where you get to listen in on a group of people playing a game of Dungeons and Dragons. There is a lot of dice rolling in this game, using a lot of different types of dice. There are dice with 4, 6, 8, 10, 12 and 20 sides that are regularly used to play the game. In combat and at other critical junctures of the game, low rolls are often bad.

The Nerd Poker players are superstitious, so before entering combat they take their dice and "roll out the ones". They start with their 20 sided dice, then do their 12 sided, 10 sided, etc. They roll all the dice of the same type at once, set aside any that show a 1, then roll the remaining dice, set aside the 1's again, and continue this process until they have rolled a 1 on every die.

For example, Brian has 3 four-sided dice. On the first roll, they show a 1, a 3 and a 3, so he sets aside the 1 and rolls the other two again. This time he gets a 1 and a 4, so he sets aside the 1 and continues to roll the last die. It takes him 3 more rolls before he gets a 1, for a total of 5 rolls. But how many rolls should Brian *expect* to have to make? Or to put it another way, if Brian rolled out the ones on his 3 four-sided dice an arbitrarily large number of times, what would be the average number of rolls required?

The input data for this question will contain 10 test cases. Each test case consists of a single line with 2 positive integers N and S where N is the number of dice and S is the number of sides on each die ($1 \leq N \times S \leq 500$). Your job is to output a line containing a single integer for each test case, representing the *expected* number of rolls required to roll out all the 1's. Your answers should always be rounded up.

Note that the sample input below contains only 3 test cases but the data files will contain 10.

Sample Input

```
1 20
125 4
25 20
```

Sample Output

```
20
20
75
```

Problem 7: CamelCase

Many programmers use CamelCase when naming variables, functions, classes and other entities. In CamelCase, when a name consists of multiple words concatenated together (such as "myawesomevariablename"), the first letter of every distinct word is capitalized. In Upper CamelCase, all words are capitalized ("MyAwesomeVariableName"). In Lower CamelCase the first word is not capitalized but the others are ("myAwesomeVariableName"). Sometimes there is more than one way to turn a string of letters into CamelCase.

The dataset for this problem will contain a dictionary, followed by 10 test cases. The dictionary starts with an integer N (where $1 \leq N \leq 200000$) followed by N lines, each containing a word. A word consists of a string of lowercase English letters. Each of the 10 lines following the dictionary will contain a single test case. Each test case consists of a string of lowercase English letters of length 2000 or less, created by concatenating words from the dictionary. Your program should output 10 integers, one per line, representing the minimum number of capitalizations required to convert each test word to Lower CamelCase, so that it can be read as a string of legal words from the dictionary.

Note that the sample input below contains only 4 test cases, but the real data files will contain 10.

<i>Sample Input</i>	<i>Sample Output</i>
26	0
aid	1
all	0
app	8
apple	
brown	
come	
country	
crab	
crabapple	
dogs	
for	
fox	
good	
is	
jumps	
lazy	
men	
now	
of	
orchard	
over	
quick	
the	
their	
time	
to	
apple	
appleorchard	
crabapple	
thequickbrownfoxjumpsoverthelazydogs	